

Generalnie uzasadnienia piszemy podobnie do tzw. ADR'ów:

<https://adr.github.io/>

<https://github.com/joelparkerhenderson/architecture-decision-record>

<https://endjin.com/blog/2024/03/adr-a-dotnet-tool-for-creating-and-managing-architecture-decision-records>

<https://github.com/npryce/adr-tools>

Opisujemy jaki problem rozwiązujemy, jakie mamy możliwości, co wybraliśmy i jakie są tego wady i zalety.

Szczególnie interesują nas wady - które zazwyczaj są pomijane. Ten format dotyczy zarówno wyboru architektury systemu, architektury aplikacji, wybranych bibliotek, sposobu testowania, narzędzi związanych z wdrożeniem.

Elementy zaznaczone na czerwono powinny zostać opisane do końca semestru 2 (PRO2).

W sekcji technicznej dokumentacji powinny się znaleźć następujące rozdziały:

1. Architektura systemu

- **Diagram C4 (poziom 1 i 2) z opisem i uzasadnieniem wybranego podejście**

2. Aplikacja serwerowa

- **Framework**
- **Architektura aplikacji**
- Wybrane biblioteki z uzasadnieniem
 - Komunikacja z bazą danych
 - Walidacja
 - Komunikacja między modułami
 - Uwierzytelnienie i autoryzacja
- Dowolne diagramy i podrozdziały, gdzie chcemy zaprezentować jakieś ciekawe rozwiązanie, moduł aplikacji, algorytm itd.

3. Aplikacja kliencka

- **Framework** - określenie głównego frameworku, który został wybrany.
- **Architektura aplikacji**
- Wybrane biblioteki z uzasadnieniem
 - Zarządzanie stanem
 - Walidacja
 - Uwierzytelnienie i autoryzacja
 - Obsługa formularzy
 - Routing
 - ...
- Dowolne diagramy i podrozdziały, gdzie chcemy zaprezentować jakieś ciekawe rozwiązanie

4. **Moduł A** - możemy dodać więcej sekcji jeśli nasza aplikacja to modularny monolit lub osobne serwisy

5. **Warstwa persystencji danych**

- Wybrana baza danych z uzasadnieniem
- Diagram encji z opisem

6. **Testowanie aplikacji**

- Wstęp
- Testy jednostkowe
- Testy integracyjne
- Testy end-to-end
- Testy wydajnościowe
- Test akceptacyjne
- ...

7. Wdrożenie aplikacji

- Architektura wdrożeniowa
- ...
- Składowanie plików

8. Podsumowanie

- ...
- Dalszy rozwój

9. Spis ilustracji

10. Spis tabel

11. Bibliografia

Ogólne uwagi

1. W dokumentacji nie interesuje nas dogłębny opis historii danego wzorca, technologii itp. Innymi słowy, nie chcemy wielu stron opisu skąd się wziął HTML. Oczekujemy, aby każdy opisał i uzasadnił swój wybór – format uzasadnienia może być podobny do tzw. ADR'ów.
2. W dokumentacji używamy formalnego języka.
3. W dokumentacji, uzasadniając wybór jakiejkolwiek technologii, staramy się podawać źródła, które potwierdzają nasze słowa.
4. Pamiętajmy o formatowaniu pracy. Rozdziały nie powinny zaczynać się pod koniec strony (wdowy i bękarty).
5. Pamiętajcie, że nie musicie uprawiać "propagandy sukcesu" w pracy. Największą wartością takiej pracy jest opisanie tego z jakimi problemami się spotykaliście i jakie złe decyzje podjęliście. Ważne, żeby pokazać, że podejmujecie przemyślane decyzje kierując się analizą wad i zalety – a nie tylko tym co obecnie jest modne.

ADR - Architecture Decision Record

ADR'y, czyli Architecture Decision Records (Rejestry Decyzji Architektonicznych), to narzędzie używane w inżynierii oprogramowania do **dokumentowania decyzji dotyczących architektury systemu**. Służą do zapisywania **istotnych wyborów technologicznych i projektowych**, jakie zostały podjęte w trakcie rozwoju systemu, oraz uzasadnienia tych wyborów.

Koncepcja ADR'ów została zaproponowana przez **Marka Richardsa w 2011 roku**. Richards zauważył, że **wiele decyzji architektonicznych nie jest dobrze dokumentowanych**, co prowadzi do problemów w utrzymaniu i rozwoju systemów informatycznych. **Dzięki ADR'om możliwe jest śledzenie i zrozumienie procesu podejmowania decyzji architektonicznych, co ułatwia zarządzanie złożonymi projektami.**

ADR'y zdobyły popularność ze względu na swoją prostotę i efektywność. Wprowadzają strukturę do dokumentacji decyzji, co jest szczególnie przydatne w dużych zespołach i projektach o długim cyklu życia. Dziś są stosowane w wielu organizacjach jako standardowa praktyka w zarządzaniu architekturą systemów.

ADR – lokalizacja

Zazwyczaj ADR najlepiej trzymać "blisko" kodu. ADR mają reprezentować faktyczny stan naszego kodu. Najlepiej zapisywać je razem z kodem aplikacji w repozytorium. Polecam skorzystać z narzędzi konsolowych, które ułatwiają generowanie ADR'ów.

- <https://github.com/joelparkerhenderson/architecture-decision-record>
- <https://endjin.com/blog/2024/03/adr-a-dotnet-tool-for-creating-and-managing-architecture-decision-records>
- <https://github.com/npryce/adr-tools>